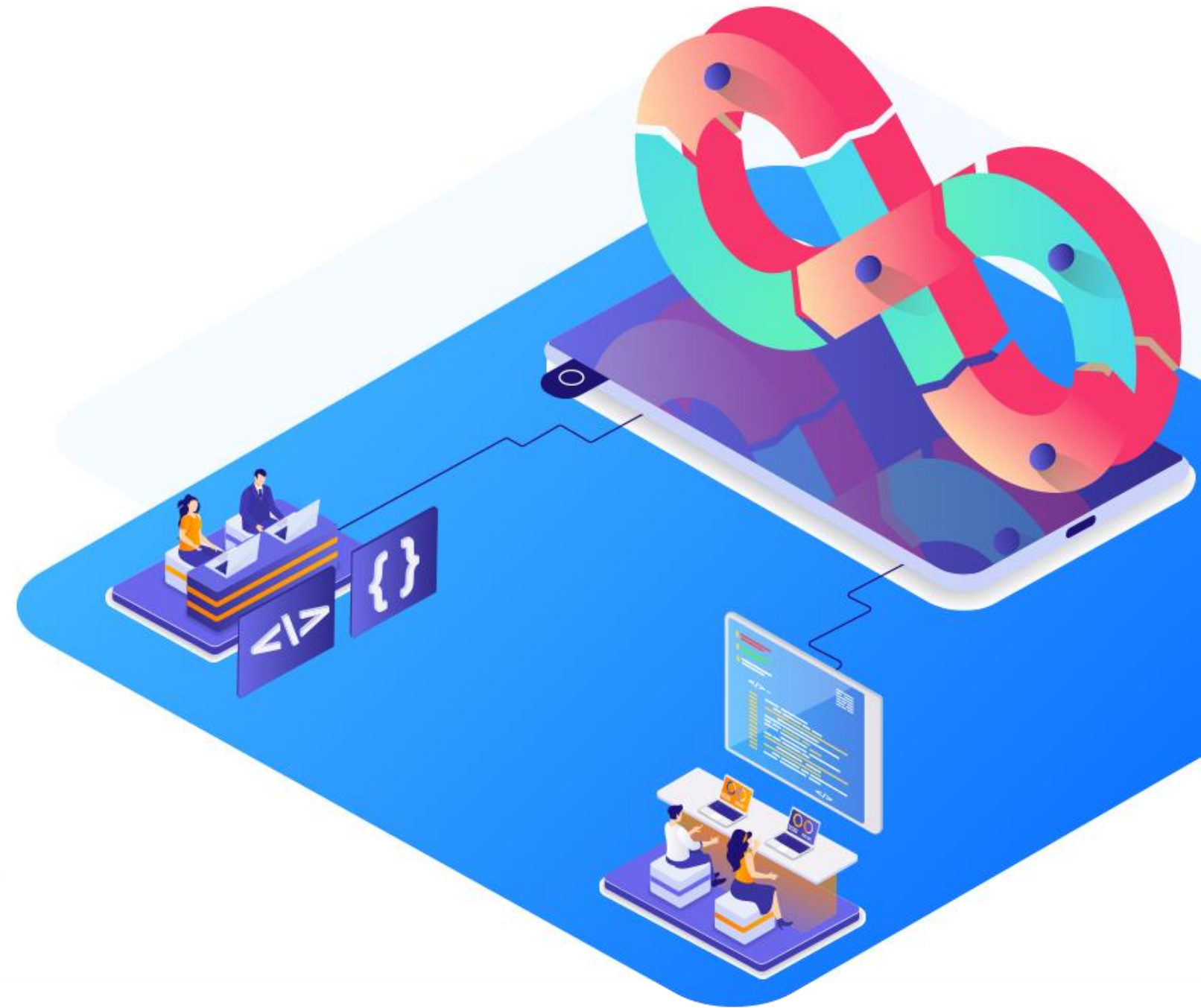


DevSecOps



Fundamentals of AWS Cloud Security



Learning Objectives

By the end of this lesson, you will be able to:

- Apply foundational security principles to ensure confidentiality, integrity, and availability of systems and data
- Configure identity and access management (IAM) policies to enforce least privilege across AWS resources
- Integrate automated security checks into the CI/CD pipeline to identify vulnerabilities during development
- Utilize AWS CloudTrail and AWS Config to identify and resolve security misconfigurations
- Apply structured troubleshooting techniques to diagnose security issues across AWS services



Security Principles



What Are Security Principles?

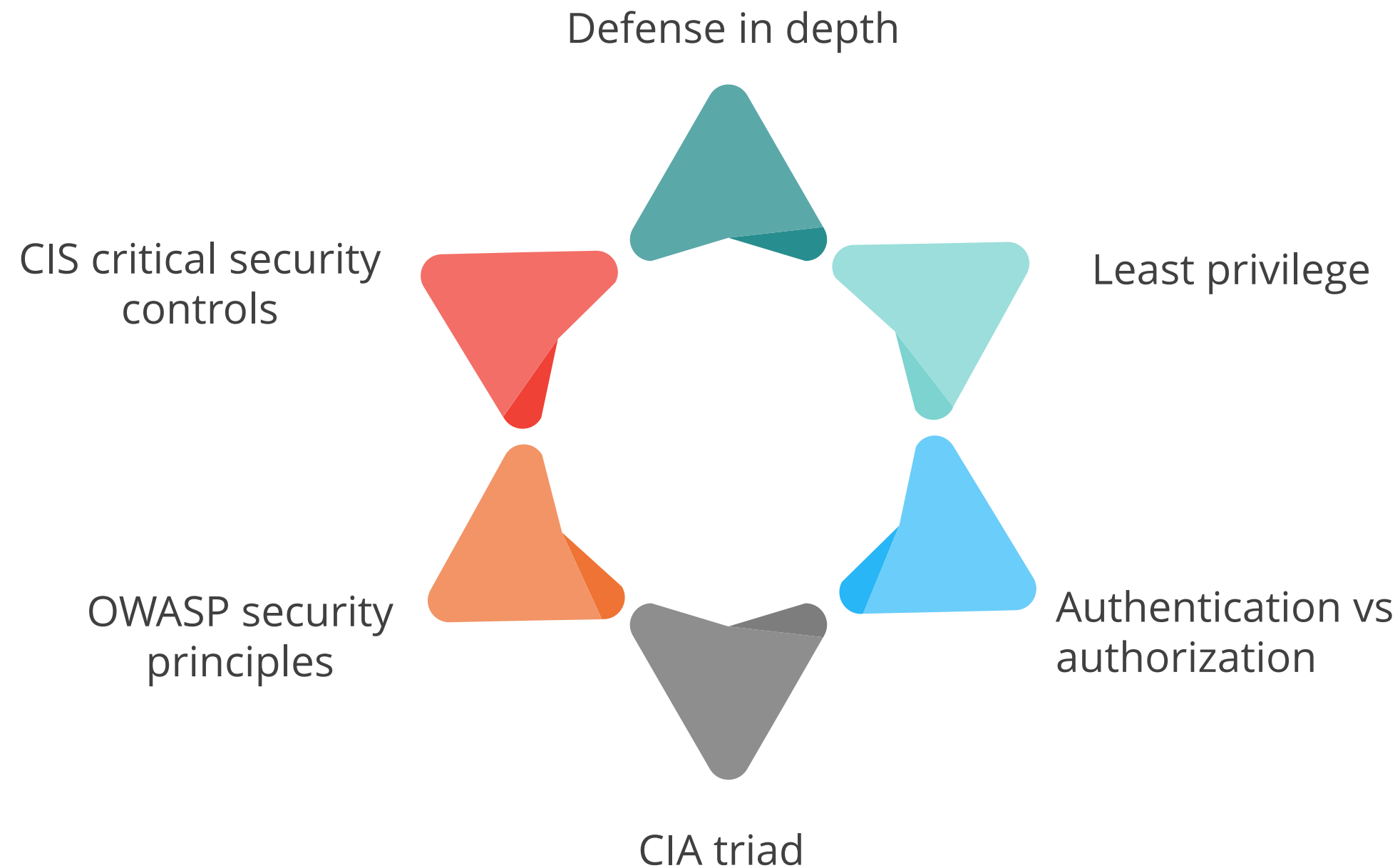
They are the guidelines created to ensure security is included in every stage of the software development lifecycle (SDLC), from planning and development to deployment and maintenance.



These principles are important for building a safe, efficient, and reliable development environment while encouraging teamwork between development, security, and operations teams.

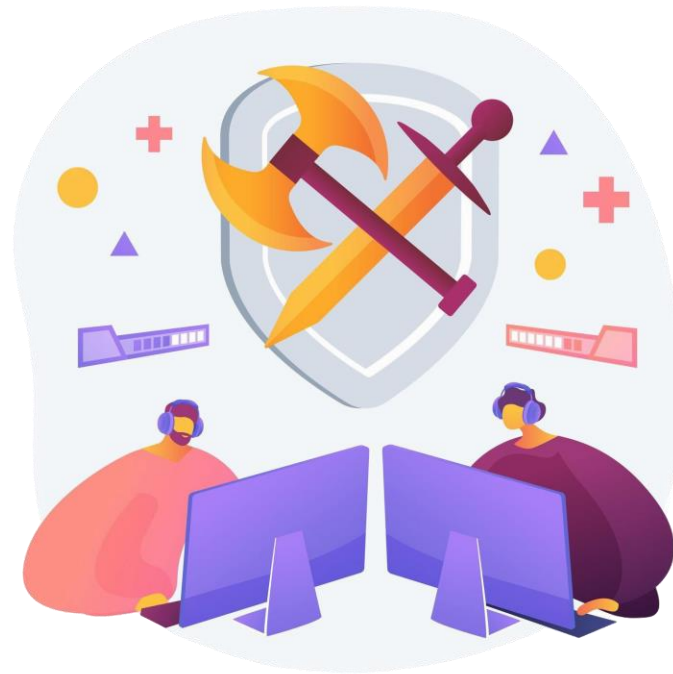
Fundamental Principles in DevSecOps Security

Below are the key DevSecOps security principles essential for building a secure and resilient development environment:



Defense in Depth

It is a layered security approach that uses multiple safeguards like firewalls, encryption, and access control to protect systems and data. If one control fails, others remain in place to mitigate threats.



The main objective of performing defense in depth is to slow down attackers and provide multiple opportunities to detect and stop them.

Least Privilege

Least privilege grants users and systems only the minimum access needed, reducing the risk of accidents or malicious activity. This limits the potential for damage by minimizing the attack surface.



The main objective of minimizing the risk of insider threats and accidental misuse is to reduce vulnerabilities and prevent security breaches.

Authentication vs Authorization

Authentication and authorization are two distinct processes that are crucial to securing systems and data, but they often work together.

Aspect	Authentication	Authorization
Definition	Confirms the identity of the entity (user/system), ensuring that only legitimate users can access a system	Determines what actions the authenticated user can take and what resources they can access
Timing	Happens before authorization, that is, during system entry	Happens after authentication, controlling access to resources
Focus	Ensures the user is legitimate (example: passwords, biometrics)	Ensures the user only accesses data or actions they're allowed to
Risk of failure	Leads to unauthorized access to the system	Leads to unauthorized access to sensitive data or actions

Quick Check



An organization is expanding its cloud infrastructure, increasing both complexity and potential attack vectors. Security teams need to prioritize defenses. What security measures would you prioritize to mitigate the greatest threats in this new cloud setup?

- A. Implement a single firewall for all systems
- B. Focus on perimeter security and ignore internal threats
- C. Prioritize defense in depth and least privilege
- D. Allow unrestricted access for faster deployment

CIA Triad

It is a foundational concept in information security that focuses on three key principles:

Confidentiality

Protects sensitive information so that only the right people have access to it, keeping it safe from unauthorized users

Integrity

Ensures that data is accurate and trustworthy, maintaining its consistency and completeness throughout its lifecycle, without being tampered with or altered

Availability

Makes sure that information and systems are available to authorized users whenever they need them, ensuring smooth and timely access

OWASP ZAP

OWASP ZAP is a free and open-source web application security scanner developed and actively maintained by a dedicated international team of developers.

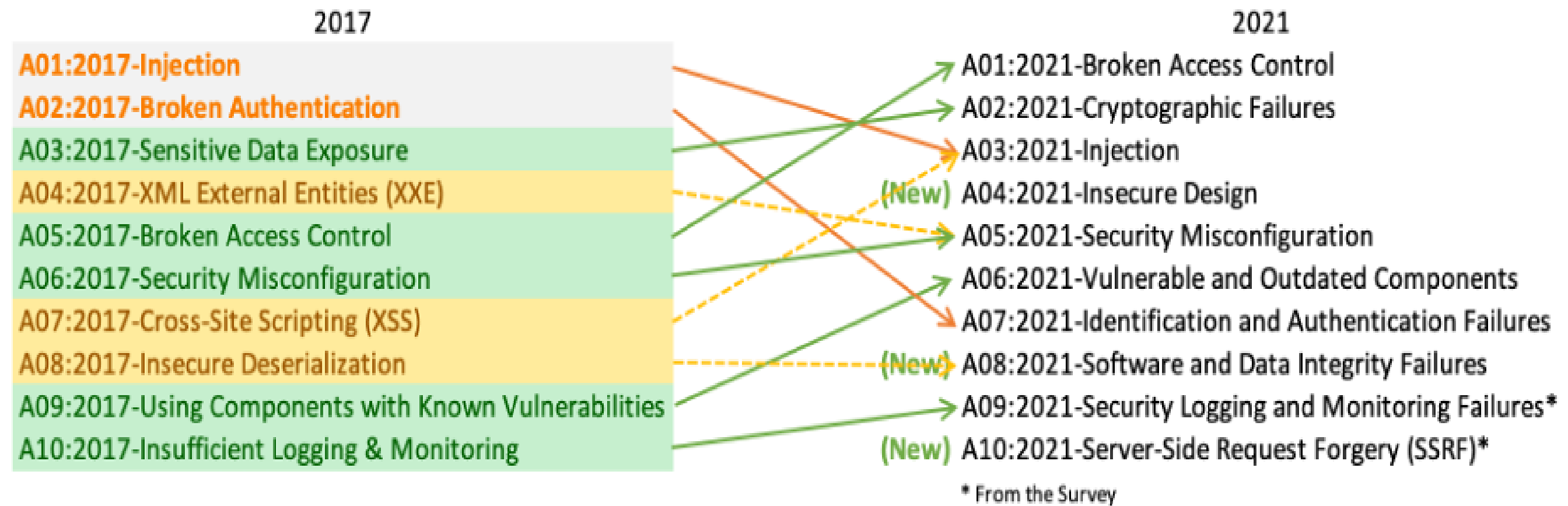


OWASP
Zed Attack Proxy

It is used to find vulnerabilities in web applications, identify misconfigurations, and test web functionality. It supports protocols like HTTP, HTTPS, SOAP, REST, FTP, and FTPS.

Top 10 OWASP Security Principles

The OWASP Top 10 Web Application Security Risks for 2021 introduces new categories and updates to address evolving threats like broken access control, cryptographic failures, and insecure design.



Top 10 OWASP Security Principles

Broken access control: Allows unauthorized users to access or modify sensitive systems due to improper permission settings

Cryptographic failures: Exposes sensitive data by using weak or incorrect encryption methods

Injection: Allows attackers to execute malicious commands by inserting untrusted data into a code interpreter

Insecure design: Fails to incorporate security principles during system design, making systems vulnerable to attacks

Security misconfiguration: Leaves systems vulnerable by incorrectly configuring settings or using outdated features

Top 10 OWASP Security Principles

Vulnerable and outdated components: Introduces security risks by using outdated or vulnerable software components without updates

Identification and authentication failures: Compromises system security by not properly verifying user identities or managing authentication

Software and data integrity failures: Fails to ensure the integrity of software updates or data, leading to potential system compromise

Security logging and monitoring failures: Misses security incidents due to insufficient logging or lack of monitoring systems

Server-side request forgery (SSRF): Allows attackers to send unauthorized requests from a server to other internal or external systems

Assisted Practice



Securing a Node.js Application Using OWASP Dependency-Check

Duration: 10 Min.

Problem statement:

You have been asked to secure a Node.js application by identifying and addressing vulnerabilities in its dependencies using OWASP Dependency-Check.

Outcome:

By the end of this demo, you will be able to run OWASP Dependency-Check to scan for vulnerabilities, analyze the findings, and implement the necessary steps to secure the Node.js application.

Note: Refer to the demo document for detailed steps:
01_Securing_a_Node.js_Application_Using_OWASP_Dependency_Check

Assisted Practice: Guidelines



Steps to be followed:

1. Configure OWASP Dependency-Check utility for Node JS application

Top 18 CIS Critical Security Controls

The CIS (Center for Internet Security) Critical Security Controls are a set of actionable recommendations designed to provide specific and effective ways to deter, detect, and defend against the most pervasive and dangerous cyber attacks.

Below are the top 18 CIS controls, a prioritized set of actions designed to help protect against cyber threats:

Inventory and control of enterprise assets

Ensures visibility and management of all hardware devices connected to the network

Inventory and control of software assets

Manages and tracks all software operating in the environment

Data protection

Safeguards sensitive data to ensure confidentiality and integrity

Top 18 CIS Critical Security Controls

Secure configuration of enterprise assets and software

Ensures systems and software are configured securely by default

Account management

Manages the life cycle of user accounts and their access

Access control management

Enforces least privilege access policies

Top 18 CIS Critical Security Controls

Continuous vulnerability management

Continuously scans for and remediates vulnerabilities

Audit log management

Logs and monitors system activities to detect security incidents

Email and web browser protections

Secures email and browser use to prevent phishing and malware

Top 18 CIS Critical Security Controls

Malware defenses

Deploys protection against malicious software threats

Data recovery

Ensures systems can recover critical data after incidents

Network infrastructure management

Protects and maintains network security and infrastructure

Top 18 CIS Critical Security Controls

Network monitoring and defense

Continuously monitors the network for unauthorized access or attacks

Security awareness and skills training

Provides ongoing security training to all personnel

Service provider management

Manages risks associated with third-party service providers

Top 18 CIS Critical Security Controls

Application software security

Ensures software is developed securely and is free of vulnerabilities

Incident response management

Plans and executes rapid response to security incidents

Penetration testing

Tests defenses regularly by simulating real-world attacks

Quick Check



An organization is handling sensitive financial data, and they need to ensure that their systems are protected from unauthorized access, tampering, and downtime. Security teams are tasked with maintaining the confidentiality, integrity, and availability of the data. What security principle should be applied to ensure data confidentiality, integrity, and availability in this scenario?

- A. Allow unrestricted access to all resources
- B. Implement the CIA triad
- C. Focus only on data encryption
- D. Prioritize performance by reducing security



Security in *AWS*

What Is AWS Security?

It is a framework of tools, services, and best practices designed by Amazon Web Services to protect cloud environments.



It ensures the safety of infrastructure, applications, and data by combining AWS's built-in security measures with customer-managed security responsibilities.

AWS Shared Responsibility Model

This model defines security roles between AWS and customers. AWS manages the cloud infrastructure, while customers secure their data and applications on it.



This shared model reduces the customers operational burden, as AWS handles and manages components from the host operating system down to the physical security of data centers.

AWS Responsibility: Security of the Cloud

This model pertains to AWS's commitment to safeguard the infrastructure that powers cloud services, including hardware, software, networking, and the physical facilities.



AWS handles the foundational security elements like the host operating system and virtualization layer, allowing customers to focus on managing their own data and application security.

Customer Responsibility: Security in the Cloud

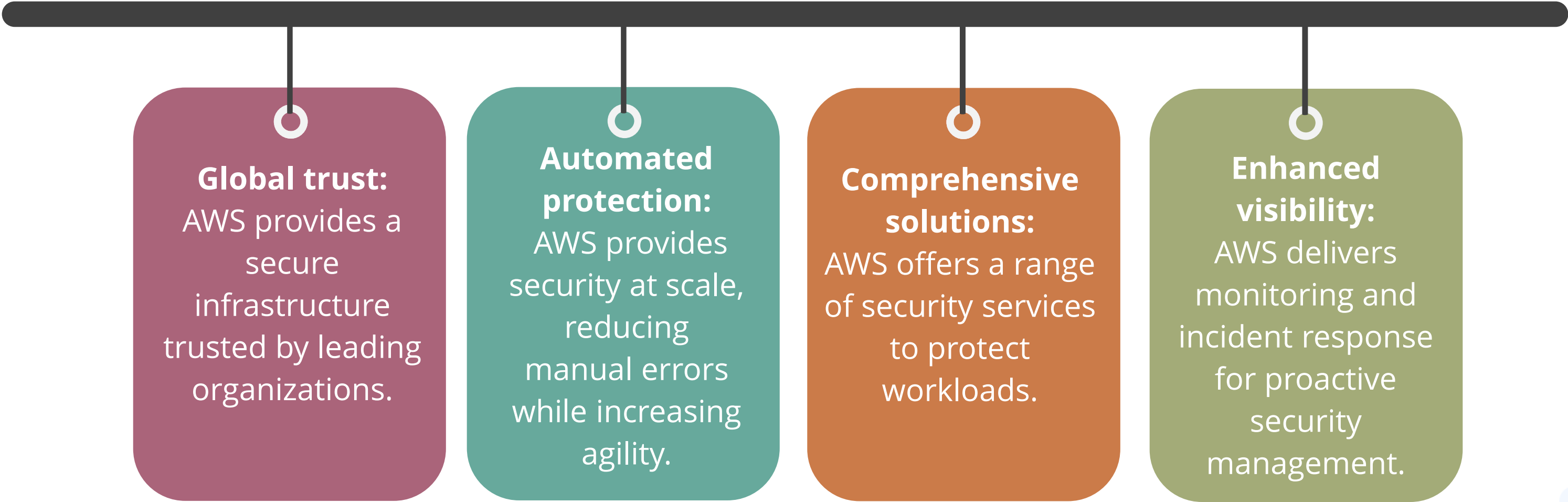
This model refers to the tasks customers must handle, which vary based on the AWS services they choose.



Customers are responsible for securing their data, managing access controls, and configuring security settings for their cloud resources, while AWS manages the underlying infrastructure.

Security in AWS: Benefits

AWS security offers a robust, comprehensive, and scalable solution to ensure your cloud workloads remain protected and compliant. The following are some key benefits:



Global trust:
AWS provides a secure infrastructure trusted by leading organizations.

Automated protection:
AWS provides security at scale, reducing manual errors while increasing agility.

Comprehensive solutions:
AWS offers a range of security services to protect workloads.

Enhanced visibility:
AWS delivers monitoring and incident response for proactive security management.

Quick Check



A company uses AWS to host its applications. Recently, a data breach occurred due to an exposed S3 bucket containing sensitive customer information. Which of the following most accurately reflects the company's responsibility under the AWS Shared Responsibility Model?

- A. Ensuring physical security of the servers where data is stored
- B. Installing security patches to the underlying server operating systems
- C. Configuring S3 buckets to ensure data is not publicly accessible
- D. Monitoring AWS data centers for unauthorized access



Security in *AWS* CodePipeline

What Is Security in AWS CodePipeline?

Security in AWS CodePipeline focuses on protecting the data, workflows, and resources involved in the pipeline against unauthorized access, misuse, and potential threats.



It involves implementing encryption, access controls, monitoring, and compliance measures.

Core Aspects of Security in AWS CodePipeline

Below are the core aspects of security in AWS CodePipeline, highlighting essential measures to safeguard data, workflows, and the pipeline environment:

01

Data protection in AWS CodePipeline

02

Identity and access management for AWS CodePipeline

03

Logging and monitoring in CodePipeline

04

Compliance validation for AWS CodePipeline

05

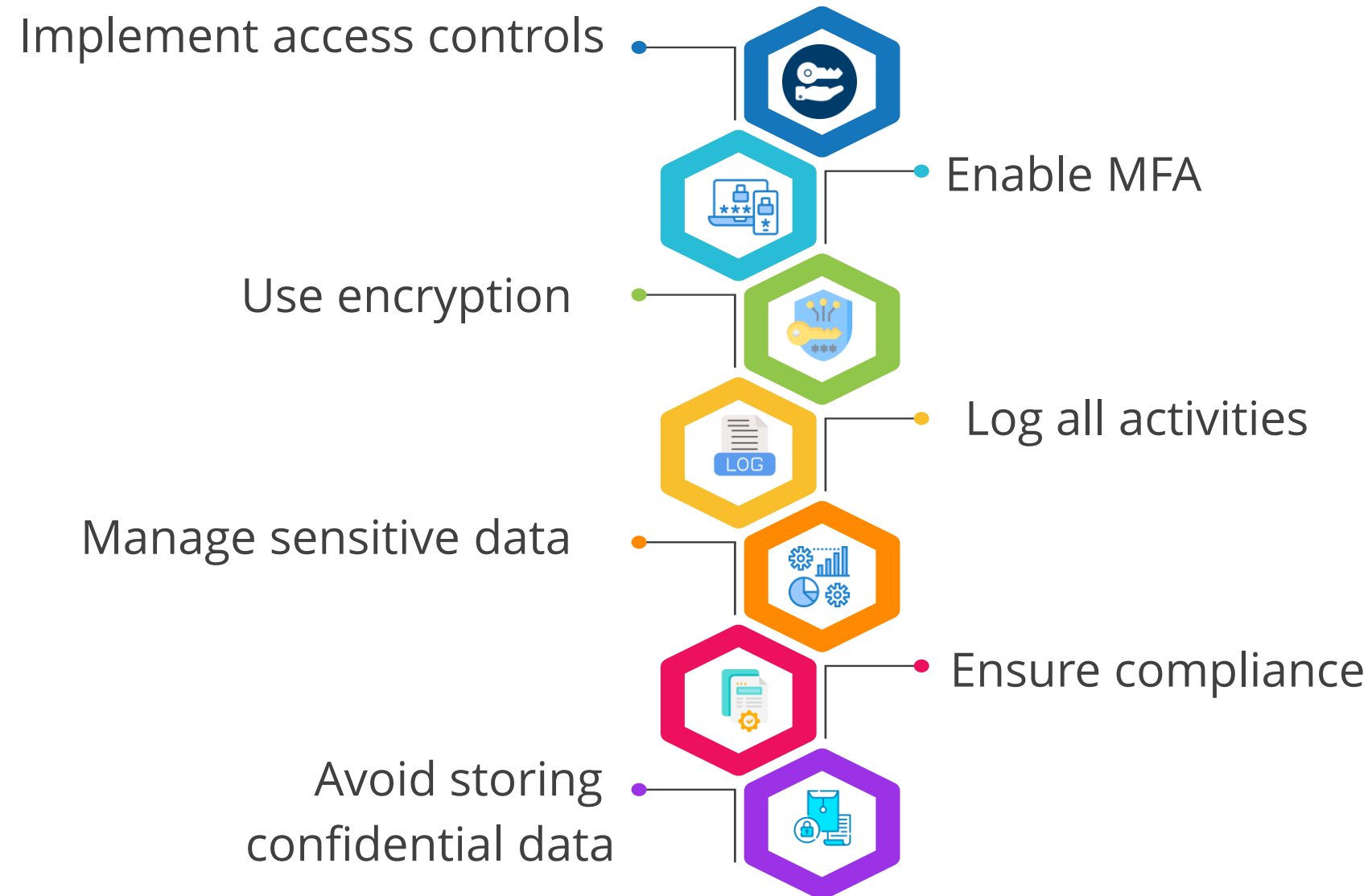
Resilience in AWS CodePipeline

06

Infrastructure security in AWS CodePipeline

Data Protection in AWS CodePipeline

AWS CodePipeline follows the shared responsibility model, where AWS secures the infrastructure, and users are responsible for controlling data access and configuring their security settings. Key practices include:



Data Protection in AWS CodePipeline

Implement access controls

It limits permissions to essential access through IAM Identity Center or IAM.

Enable MFA

It requires MFA for additional security on user accounts.

Use encryption

It secures data by encrypting it, protecting sensitive information during storage and transmission.

Log all activities

It monitors user actions by logging activity, helping track access and detect suspicious behavior.

Data Protection in AWS CodePipeline

Manage sensitive data

It utilizes Amazon Macie to protect sensitive data, particularly in Amazon S3.

Ensure compliance

It ensures federal compliance by using FIPS 140-3 endpoints as needed.

Avoid storing confidential data

It uses AWS config rules to restrict sensitive data in tags and name fields, minimizing log exposures.

Identity and Access Management for AWS CodePipeline

AWS Identity and Access Management (IAM) lets administrators control access to CodePipeline by managing user permissions. Key IAM topics for CodePipeline include:

Authentication management

Ensures users and services are correctly authenticated to access CodePipeline resources

Policy-based access control

Sets granular permissions to control user and resource actions

Role management in CodePipeline

Demonstrates applying identity-based and resource-based roles for efficient access management

Access issue troubleshooting

Helps diagnose and resolve access-related issues, maintaining secure operations

Logging and Monitoring in CodePipeline

In CodePipeline, monitoring and logging enable proactive tracking of system events and user actions to enhance security and streamline troubleshooting.

Logging and monitoring help track user actions and resource events using CloudTrail for API logs and CloudWatch for alerts, enhancing security and troubleshooting.

AWS CloudTrail

Logs API calls and events made by or on behalf of an AWS account, helping track access and actions in CodePipeline



Amazon CloudWatch events

Monitors resources and applications in AWS, allowing users to set alerts based on custom metrics for proactive monitoring

Compliance Validation for AWS CodePipeline

Ensuring compliance in AWS CodePipeline requires leveraging AWS tools to meet industry standards. AWS provides the following resources to help with compliance:

AWS artifact

It provides access to third-party audit reports to verify compliance for AWS services.

Quick start guides

It offers step-by-step guidance for deploying compliant, security-focused environments.

AWS compliance resources

It includes guides and workbooks tailored to industry standards and regional requirements.

Compliance Validation for AWS CodePipeline

AWS provides the following resources to help with compliance:

AWS config

It evaluates AWS resources against best practices and regulations, helping validate compliance configurations.

Security hub and GuardDuty

These services offer a comprehensive security view, detecting threats and checking compliance against industry standards.

AWS audit manager

It assists with continuous auditing to manage compliance across regulatory frameworks.

Resilience in AWS CodePipeline

Ensuring resilience in AWS CodePipeline leverages AWS's robust global infrastructure.
Key strategies include:

Leverage AWS regions and availability zones

It is used to distribute workloads across multiple zones, enhancing high availability and fault tolerance.

Enable automatic failover

It is used to configure systems to automatically switch zones during failures, ensuring uninterrupted service.

Utilize low-latency networking

It is used to take advantage of AWS's high-speed network, improving performance across regions.

Resilience in AWS CodePipeline

Key strategies include:

Design for scalability

It is used to build scalable architectures that handle variable workloads seamlessly.

Ensure fault tolerance

It is used to deploy resources redundantly across zones, supporting quick recovery from failures.

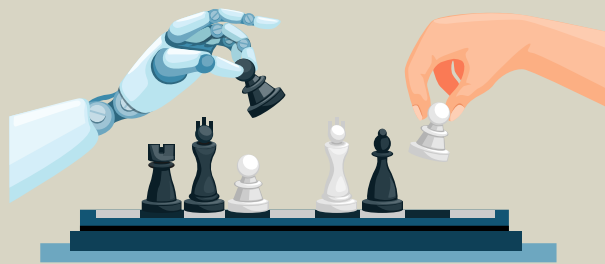
Infrastructure Security in AWS CodePipeline

AWS CodePipeline leverages AWS's global network security for infrastructure protection and requires secure communication standards for client access. Key practices include:

-  **Secure network access:** It uses TLS (minimum TLS 1.2, recommended TLS 1.3) to ensure encrypted communication with CodePipeline.
-  **Strong cipher suites:** It requires support for cipher suites with perfect forward secrecy (PFS) like DHE and ECDHE, enhancing data protection.
-  **Request signing:** It mandates requests be signed using an access key ID and secret key or temporary credentials from AWS Security Token Service (STS) for secure authentication.

Use Case

Securing DevOps Pipelines for Online Banking Applications



Challenge

- The financial institution faced risks of data breaches and unauthorized access while developing an online banking application.
- Early-stage vulnerabilities in the DevOps pipeline posed a threat to sensitive financial data and regulatory compliance.
- The lack of proactive security measures could lead to significant financial and reputational damage.

Use Case

Securing DevOps Pipelines for Online Banking Applications



Solution

- Adopted threat modeling frameworks like STRIDE or PASTA to identify risks and vulnerabilities in the pipeline
- Implemented encryption for sensitive data, multi-factor authentication (MFA), and robust access controls using AWS IAM
- Enabled real-time activity monitoring and auditing with tools such as AWS CloudTrail

Use Case

Securing DevOps Pipelines for Online Banking Applications



Outcome

- Mitigated risks of data breaches and unauthorized access, ensuring a secure DevOps pipeline
- Achieved compliance with financial regulations, reducing potential legal liabilities
- Built customer trust and confidence by protecting sensitive data and delivering a secure banking application

Assisted Practice



Implementing Encryption for Code Pipeline Workflow

Duration: 10 Min.

Problem statement:

You have been tasked with implementing encryption for an AWS CodePipeline workflow to secure data in transit and at rest.

Outcome:

By the end of this demo, you will be able to configure encryption settings for CodePipeline, ensuring compliance with security standards and protecting sensitive information.

Note: Refer to the demo document for detailed steps:
[02_Implementing_Encryption_for_Code_Pipeline_Workflow](#)

Assisted Practice: Guidelines



Steps to be followed:

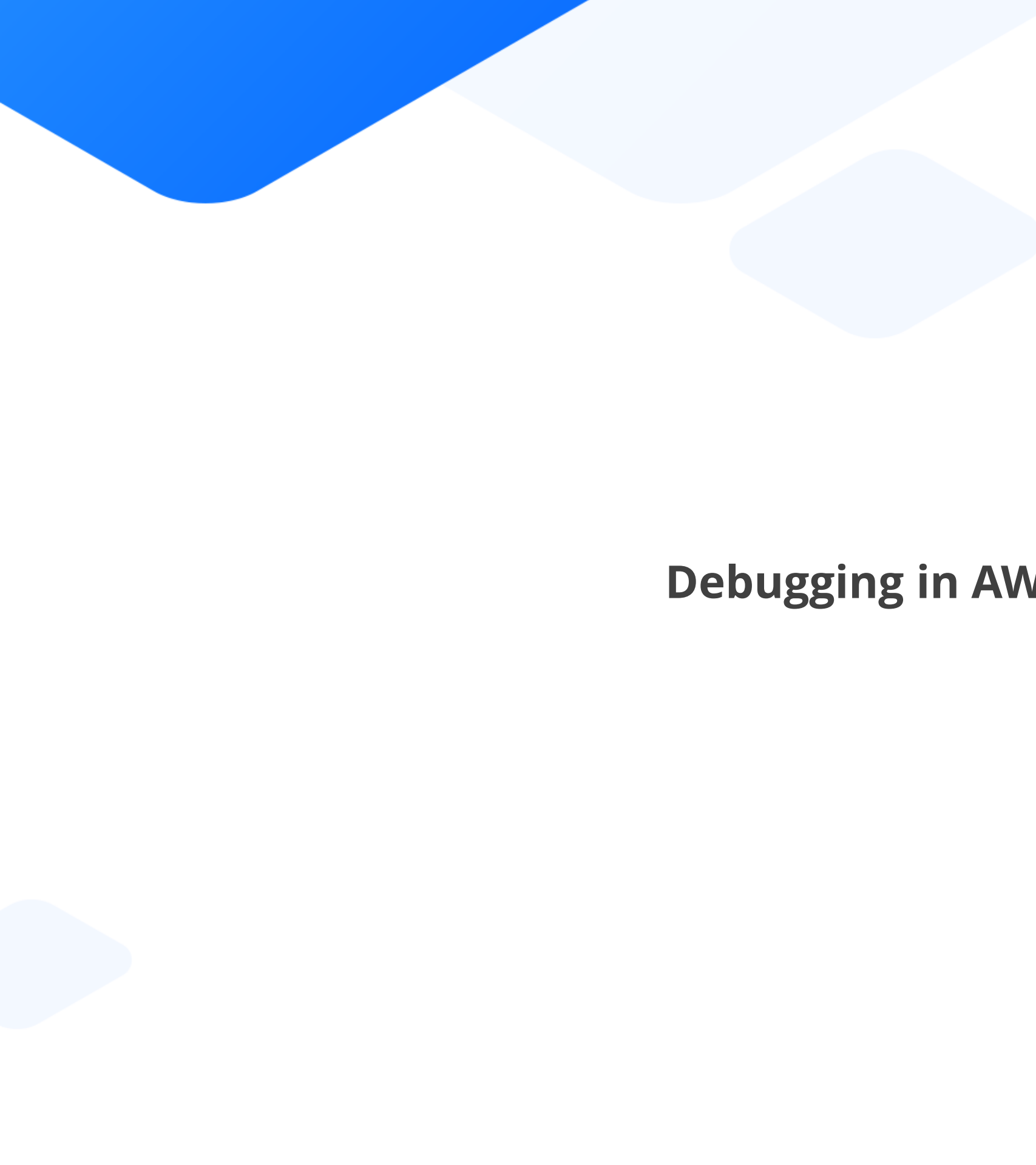
1. Create a S3 bucket with a KMS key
2. Create a Continuous Integration pipeline
3. Edit the Pipeline and add the Artifacts section
4. Test the updated pipeline

Quick Check

A company is implementing AWS CodePipeline to improve the deployment process while ensuring strong security controls. Which action would most effectively enhance the security of the pipeline?

- A. Use long-term credentials for all service interactions within CodePipeline
- B. Integrate AWS IAM roles with CodePipeline to manage permissions for individual resources securely
- C. Set up CloudTrail and CloudWatch to monitor user actions and detect unusual activity in real time
- D. Skip regular compliance checks to expedite the deployment process





Debugging in AWS Cloud Security

What Is Debugging in AWS Cloud Security?

It refers to the process of identifying, analyzing, and resolving security-related issues within cloud infrastructure and services on AWS.



It involves using various tools and techniques to detect vulnerabilities, misconfigurations, or policy violations, ensuring that the cloud environment remains secure and compliant with security standards.

Importance of Debugging in AWS Cloud Security

Below are the key reasons highlighting the importance of debugging in AWS cloud security:

Ensure secure configurations

Debugging helps verify that cloud configurations are set up securely to prevent vulnerabilities and unauthorized access.

Detect and resolve threats early

It identifies potential security issues or breaches in their early stages, allowing for quick resolution before they escalate.

Support compliance requirements

It ensures cloud infrastructure adheres to industry security standards and compliance regulations.

Importance of Debugging in AWS Cloud Security

Below are the key reasons highlighting the importance of debugging in AWS cloud security:

Maintain service availability

It prevents security issues that could disrupt services, ensuring continuous availability and reliability.

Strengthen incident response

It improves the speed and efficiency of responding to security incidents, minimizing potential damage and recovery time.

Tools for Debugging in AWS Environments

The following are key tools for debugging in AWS environments:



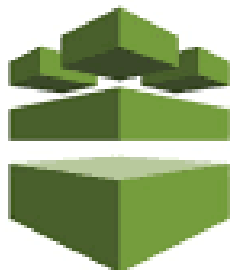
Amazon
CloudWatch

Monitors AWS environments, tracks metrics, checks logs, and sets alarms to spot issues in real time



AWS CloudTrail

Logs all API calls and user actions to help security teams trace and debug suspicious activity



AWS Config

Tracks and records configuration changes to find misconfigurations and fix compliance issues

Tools for Debugging in AWS Environments

The following are key tools for debugging in AWS environments:



Amazon GuardDuty

Analyzes logs with machine learning to find threats and help debug unusual activities quickly



Traces issues in distributed applications, showing a visual map to pinpoint and fix problem areas



AWS Systems
Manager

Offers insights, automation, and remote management to help debug infrastructure problems across AWS resources

Debugging IAM Permissions and Policies

It involves checking user, group, and role permissions to find and fix access issues or security risks.

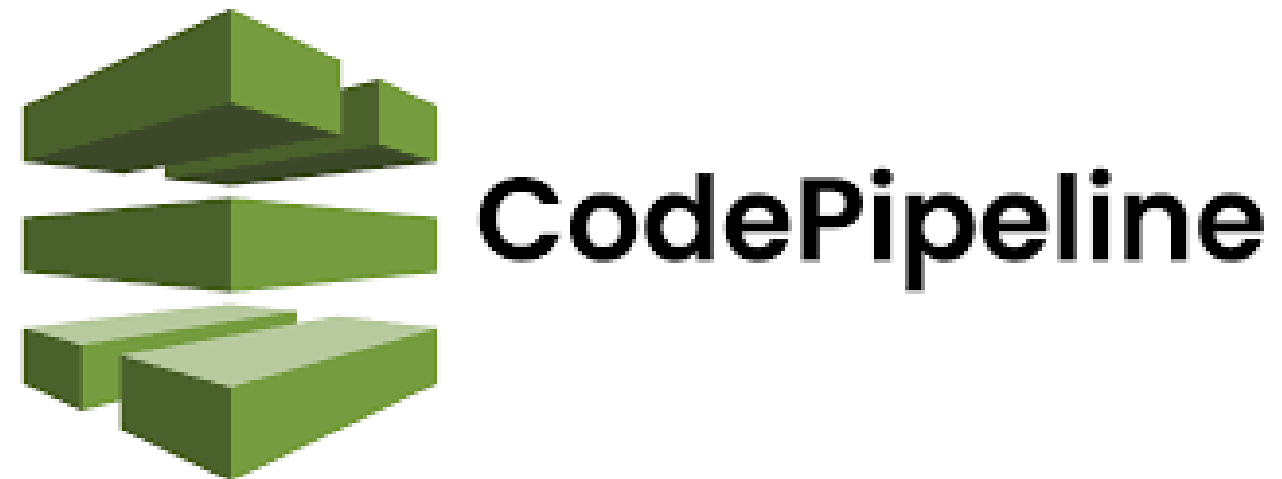


AWS IAM

Debugging IAM permissions and policies is important for secure access control in AWS. Common problems include overly permissive policies, misconfigured roles, and access denials.

Debugging Issues in AWS CodePipeline

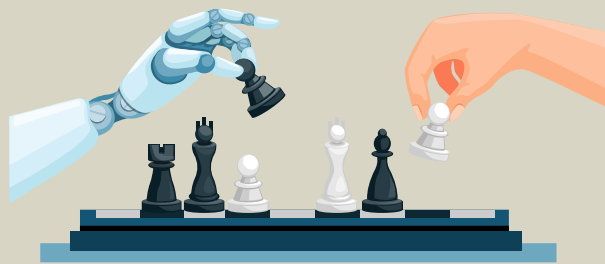
Debugging problems in AWS CodePipeline is vital to keep CI/CD workflows running smoothly. Issues can occur at any stage, such as source, build, test, or deployment, and can disrupt the pipeline.



Quickly finding and fixing these issues ensures the pipeline remains efficient and reliable, helping prevent deployment delays and keeping the workflow running smoothly.

Use Case

Identifying Misconfigurations in AWS Security Groups



Challenge

- An organization faced vulnerabilities caused by overly permissive rules and unintended open access in AWS security groups.
- They experienced increased risks of unauthorized access and potential data breaches due to misconfigurations.
- Additionally, they lacked automated tools to effectively monitor, detect, and resolve security group issues.

Use Case

Identifying Misconfigurations in AWS Security Groups



Solution

- Used AWS Config to track security group configurations and identify non-compliant rules
- Leveraged AWS CloudTrail to log changes and access attempts for actionable insights
- Generated detailed reports to debug and resolve misconfigurations efficiently

Use Case

Identifying Misconfigurations in AWS Security Groups



Outcome

- Identified and corrected misconfigurations to reduce security risks and improve reliability
- Streamlined the debugging process with detailed logs and reports, enabling quick resolutions
- Achieved compliance with security standards to ensure secure deployments and reduce vulnerabilities

Debugging in DevSecOps vs. Cloud Security

Feature	Debugging in DevSecOps	Debugging in cloud security
Scope and integration	Integrates security throughout the SDLC	Focuses on securing cloud-based resources and infrastructure
Continuous improvement	Utilizes automated tools in CI/CD pipelines for continuous testing and feedback	Addresses dynamic scaling and resource management in the cloud
Collaboration	Requires collaboration across development, operations, and security teams.	Involves cloud-specific roles and expertise, such as cloud security architects
Environment specificity	Considers generic software development environments	Tailored to specific cloud environments
Access and control	Manages security as part of the development process	Focuses and identifies permissions on cloud-specific access controls
Tools and practices	Employs SAST, DAST, IAST, and SIEM systems	Uses CSPM, CWPPs, CASBs, and IAM systems specific to cloud platforms

Assisted Practice



Validating Misconfigurations in Security Groups

Duration: 10 Min.

Problem statement:

You have been tasked with identifying and validating misconfigurations in security groups to ensure compliance with best practices and enhance the security of the cloud environment.

Outcome:

By the end of this demo, you will be able to analyze security group configurations, detect potential misconfigurations, and apply corrective measures to secure resources effectively.

Note: Refer to the demo document for detailed steps:
03_Validating_Security_Groups_misconfigurations

Assisted Practice: Guidelines



Steps to be followed:

1. Create and configure a security group
2. Validate security configurations using AWS Trusted Advisor checks

Assisted Practice



Debugging Using AWS CloudTrail

Duration: 10 Min.

Problem statement:

You have been asked to debug issues in your AWS environment by analyzing activity logs using AWS CloudTrail.

Outcome:

By the end of this demo, you will be able to use AWS CloudTrail to identify, analyze, and resolve issues by accessing detailed logs of API activity and changes in your AWS account.

Note: Refer to the demo document for detailed steps:
04_Debugging_Using_AWS_CloudTrail

Assisted Practice: Guidelines



Steps to be followed:

1. Create a CloudTrail
2. Validate CloudTrail logs

Quick Check



A cloud operations team needs to ensure their AWS environment is secure and properly configured. They want to track all API calls, monitor changes to resource configurations, and get alerts for any potential issues. Which combination of tools would best help them achieve these goals?

- A. AWS Lambda and Amazon S3
- B. Amazon RDS and AWS Trusted Advisor
- C. Amazon DynamoDB and AWS Shield
- D. AWS CloudWatch, AWS CloudTrail, and AWS Config

Key Takeaways

- Defense in depth is a layered security approach that uses multiple safeguards like firewalls, encryption, and access control to protect systems and data.
- Confidentiality protects sensitive information so that only the right people can access it, keeping it safe from unauthorized users.
- Security of the cloud model pertains to AWS's commitment to safeguarding the infrastructure that powers cloud services, including hardware, software, networking, and physical facilities.
- AWS security offers a robust, comprehensive, and scalable solution to ensure the cloud workloads remain protected and compliant.
- Debugging IAM permissions and policies involves checking user, group, and role permissions to find and fix access issues or security risks.



Automating Security Monitoring for AWS IAM Access Key Creation

Duration: 25 Min.

Project agenda: To implement automated security monitoring for AWS IAM access key creation using AWS CloudTrail, Lambda, DynamoDB, and EventBridge. This ensures enhanced security by detecting and logging access key creation events and storing them in DynamoDB for auditing and tracking.

Description: You are a cloud engineer tasked with securing AWS resources by monitoring IAM access key creation events. The goal is to set up a system where AWS CloudTrail captures the creation of access keys, triggers an AWS Lambda function via EventBridge, and logs the event details into a DynamoDB table. This solution aims to identify potential security risks early, track malicious activity, and improve the overall security posture of the AWS account.



Automating Security Monitoring for AWS IAM Access Key Creation

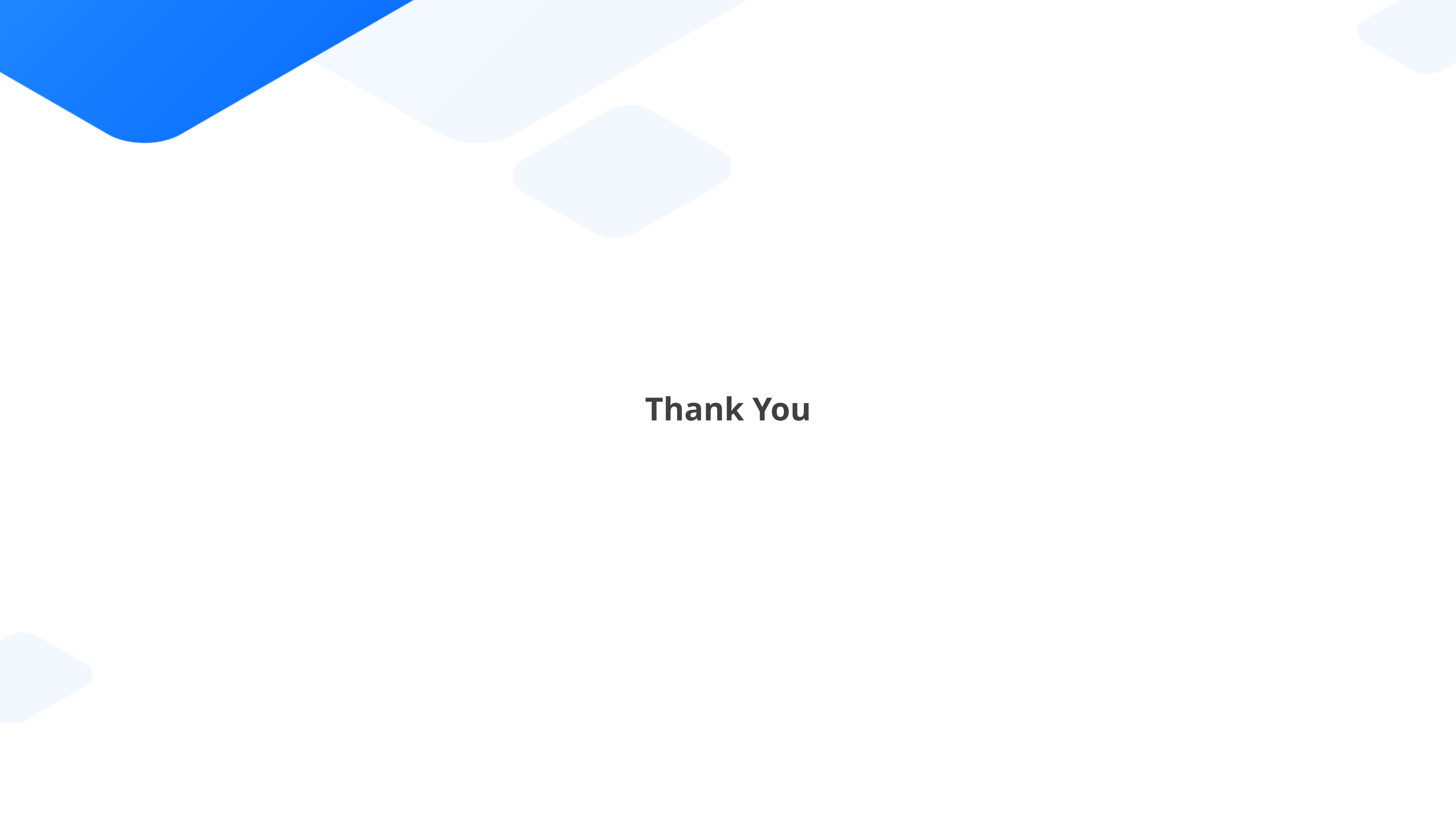
Duration: 25 Min.

Perform the following:

1. Create a CloudTrail to log IAM activities
2. Create and deploy a Lambda function to track access key creation
3. Create a DynamoDB table to store CloudTrail event IDs
4. Set up an EventBridge rule to trigger the Lambda function
5. Generate access and secret keys in IAM for testing
6. Monitor Lambda execution and CloudTrail logs in CloudWatch

Expected deliverables: A security monitoring system with CloudTrail capturing key creation events, a Lambda function logging them to DynamoDB, an EventBridge trigger, and CloudWatch logs for validation





Thank You